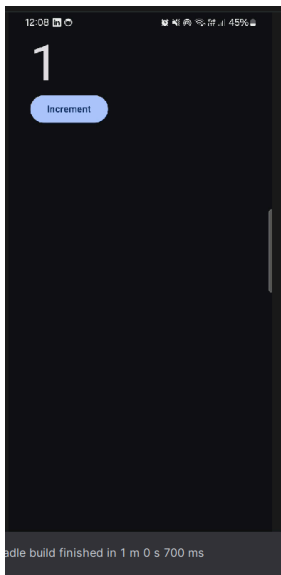


State Hoisting

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.
 - a. Apa yang terjadi setiap kali tombol "Increment" diklik?



Angka pada tampilan Text bertambah 1 setiap kali tombol diklik ($0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$).

- b. Apa yang menyebabkan hal tersebut terjadi?

Ketika tombol diklik, perintah `number++` mengubah nilai variabel `number`. Karena `number` adalah variabel state (`mutableStateOf`), Jetpack Compose otomatis melakukan rekomposisi (menggambar ulang UI). Fungsi `remember` berfungsi menjaga nilai `number` terbaru tetap tersimpan di memori.

- c. Variabel `state` apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

```
var number by remember { mutableStateOf( value = 0) }  
val label by remember { mutableStateOf( value = "Increment") }
```

- d. Variabel `state` apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

```
val label by remember { mutableStateOf( value = "Increment") }
```

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable `CounterScreen` agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Mengubah Label Tombol:

Ubah nilai inisialisasi state label dari "Increment" menjadi "Increment 2".

```
var number by remember { mutableStateOf( value = 0) }  
val label by remember { mutableStateOf( value = "Increment 2") }
```

Mengubah Logika Increment (Bertambah 2):

Ubah callback onClick pada komponen Button dari `number++` menjadi `number += 2` (pastikan menggunakan operator assignment `+=` agar variabel `number` diperbarui).

```
Button(onClick = { number += 2 }) {  
    Text(text = "$label")  
}
```

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.



3. Perhatikan bahwa deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().



```

@Composable
fun CounterScreen(
    modifier: Modifier = Modifier,
    number: Int,
    label: String,
    onClick: () -> Unit
) {
    // var number by remember { mutableStateOf(0) }
    // val label by remember { mutableStateOf("Increment") }
    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
        Button(onClick = onClick) {
            Text(text = "$label")
        }
    }
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        StatehoistingTheme {
            Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                var number by remember { mutableStateOf( value = 0) }
                CounterScreen(
                    Modifier.padding( all = 32.dp),
                    number = number,
                    label = "Increment",
                    onClick = { number++ }
                )
            }
        }
    }
}

```

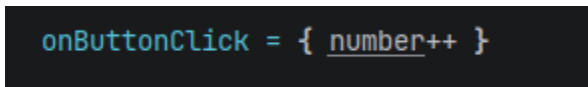
4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

- Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Ya, berjalan dengan baik dan menampilkan hasil yang sama persis.

Meskipun struktur kodenya telah diubah melalui State Hoisting (pemisahan logika state dari UI), perilaku UI bagi pengguna tetap sama: angka bertambah 1 setiap kali tombol "Increment" diklik.

- Di bagian kode program manakah saat ini proses increment dilakukan?

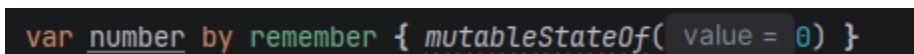


```
onClick = { number++ }
```

- Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State Composable Text (number):

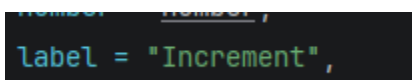
Dideklarasikan di dalam fungsi onCreate() sebelum pemanggilan CounterScreen:



```
var number by remember { mutableStateOf( value = 0) }
```

State/Nilai Label Composable Button ("Increment"):

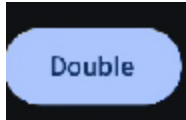
Diteruskan langsung dari fungsi onCreate() melalui argumen parameter label:



```
label = "Increment",
```

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

- Label tombol "Increment" menjadi "Double"



- b. Nilai awal variabel `number` dari 0 menjadi 1.

```
var number by remember { mutableStateOf( value = 1) }  
CounterScreen(
```

1

- c. Ubah fungsi/lambda `onButtonClick` pada pemanggilan Composable `CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.

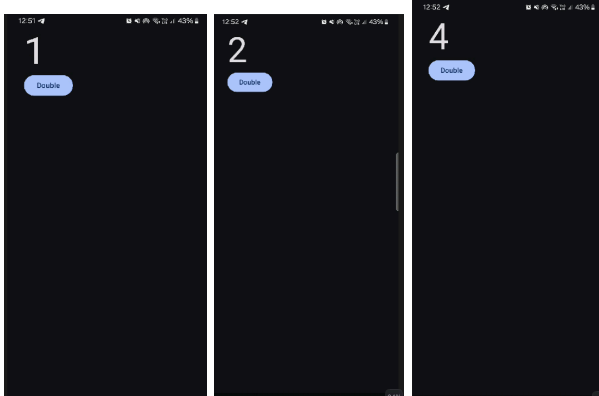
```
onButtonClick = { number *= 2 }
```

- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    enableEdgeToEdge()  
    setContent {  
        StatehoistingTheme {  
            Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                var number by remember { mutableStateOf( value = 1) }  
                CounterScreen(  
                    Modifier.padding( all = 32.dp),  
                    number = number,  
                    label = "Double",  
                    onButtonClick = { number *= 2 }  
                )  
            }  
        }  
    }  
}
```

6. Jalankan program,

- a. Perhatikan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel `number` digandakan.



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Tahap 7 (State Hoisting)

Perubahan dilakukan pada dua bagian di MainActivity.kt:

1. Fungsi CounterScreen:

Diubah menjadi stateless dengan menambahkan parameter number, label, dan onClick, serta menghapus variabel state internal di dalam body fungsinya.

2. Fungsi onCreate():

Menambahkan deklarasi state var number by remember { mutableStateOf(0) } di dalam setContent dan meneruskannya ke fungsi CounterScreen.

Tahap 8 (Modifikasi ke Double Counter)

Perubahan seluruhnya dilakukan di satu bagian, yaitu pada fungsi onCreate() di dalam blok setContent:

- Mengubah nilai awal state number dari 0 menjadi 1 (mutableStateOf(1)).
- Mengubah parameter label dari "Increment" menjadi "Double".
- Mengubah lambda callback onClick dari { number++ } menjadi { number *= 2 }.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

PADA TAHAP 7 ada perubahan di dalam fungsi CounterScreen:

1. Mengubah Signature Fungsi: Parameter fungsi ditambahkan agar menerima data dan event dari luar:

```
fun CounterScreen(
    modifier: Modifier = Modifier,
    number: Int,
    label: String,
    onClick: () -> Unit
) {
    // var number by remember { mutableStateOf(0) }
    // val label by remember { mutableStateOf("Increment") }
    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
        Button(onClick = onClick) {
            Text(text = "$label")
        }
    }
}
```

2. Menghapus State Lokal: Deklarasi `var number` dan `val label` di dalam body `CounterScreen` dihapus/di-comment agar fungsinya menjadi stateless.
3. Mengubah `OnClick` Button: Parameter `onClick` pada Button diubah dari `{ number++ }` menjadi panggilan callback `{ onButtonClick }`.

PADA TAHAP 8 TIDAK ADA perubahan sama sekali di dalam fungsi `CounterScreen`.

Mengapa ?

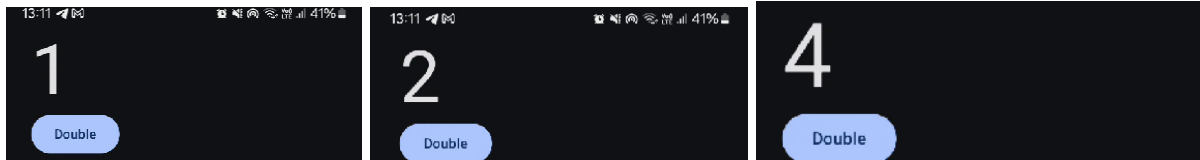
Karena `CounterScreen` sudah diubah menjadi stateless pada Tahap 7, seluruh perubahan logika pada Tahap 8 (angka awal 1, label "Double", dan perkalian x2) dilakukan di luar `CounterScreen`, yaitu di dalam fungsi `onCreate()`. `CounterScreen` hanya menerima nilai tersebut melalui parameternya.

Maksudnya adalah State Hoisting membuat komponen UI bersifat Stateless (tanpa state internal).

Perubahan Konfigurasi

Aktivitas Percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.
Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

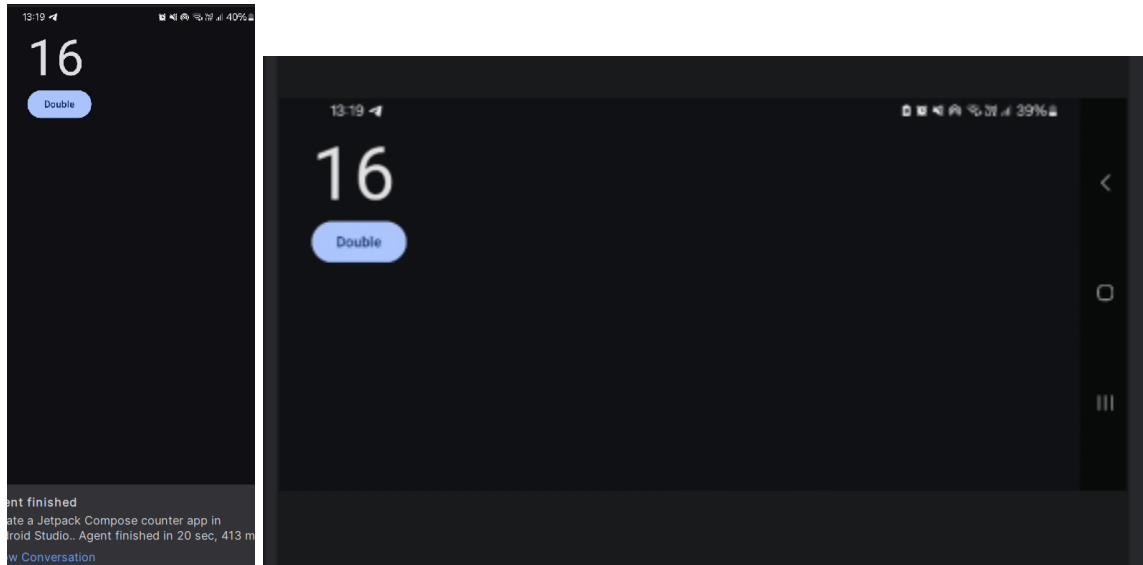


2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape. 1 (ke reset)
 - b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
Hal ini terjadi karena adanya Perubahan Konfigurasi (Configuration Change) akibat rotasi layar dari portrait ke landscape:
 - Destroy & Recreate Activity:
Saat layar diputar, Android secara default mematikan (destroy) dan membuat ulang (recreate) MainActivity.
 - Keterbatasan remember:
Fungsi remember { mutableStateOf(...) } hanya mempertahankan state selama proses rekomposisi dalam satu lifecycle Activity yang sama. State ini tidak tersimpan jika Activity dihancurkan akibat perubahan konfigurasi.
 - Inisialisasi Ulang:
Karena Activity dibuat ulang dari awal, fungsi onCreate() dijalankan kembali sehingga variabel number terinisialisasi ulang ke nilai awalnya.

```
Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
    var number by rememberSaveable { mutableStateOf( value = 1) }
    CounterScreen(
        Modifier.padding( all = 32.dp),
        number = number,
        label = "Double",
        onClick = { number *= 2 }
    )
}
```

3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Karena penggunaan `rememberSaveable` secara otomatis menyimpan nilai variabel number ke dalam Bundle (Saved Instance State). Saat layar diputar dan Activity dibuat ulang, Jetpack Compose memulihkan (restore) nilai state tersebut dari Bundle, sehingga angka tidak ter-reset ke 1.

- c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

- `remember`
Hanya menyimpan state selama proses rekomposisi (recomposition) pada lifecycle Activity yang sama. Nilai state akan hilang/reset jika terjadi perubahan konfigurasi (seperti rotasi layar) atau saat Activity dibuat ulang.
- `rememberSaveable`
Menyimpan state yang dapat bertahan melintasi perubahan konfigurasi (seperti rotasi layar dari portrait ke landscape) dan process death, karena datanya disimpan ke dalam Bundle mekanisme saved instance state.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh Kasus Nyata:

- Form Input Teks (Nama, Email, Alamat):
Teks yang sedang diketik pengguna tidak hilang/terhapus saat layar HP diputar.
- Filter Produk E-Commerce:
Pilihan filter (kategori, harga, rating) tidak ter-reset saat posisi layar berubah.
- Toggle Sembunyikan/Lihat Password:
Status ikon "mata" pada kolom kata sandi tetap konsisten saat rotasi layar.

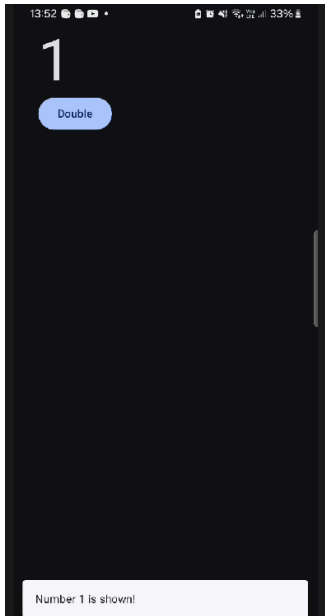
Manfaat Utama:

- Mencegah Kehilangan Data: Menjaga data sementara pengguna agar tidak hilang akibat rotasi layar atau perubahan konfigurasi.
- Meningkatkan UX: Pengguna tidak perlu mengetik atau memilih ulang dari awal, sehingga aplikasi terasa lebih lancar dan nyaman digunakan.

LaunchedEffect

Aktivitas Percobaan

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?



- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Karena blok `LaunchedEffect` secara otomatis dieksekusi oleh Jetpack Compose ketika komponen pertama kali dirender (initial composition).

Pada awal aplikasi berjalan, variabel `number` bernilai 1. Karena `LaunchedEffect` dipanggil pada awal pembentukan UI, pesan `Snackbar` langsung muncul tanpa menunggu aksi tombol dari pengguna.

- c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana? Pemicunya:

Variabel `number` yang dijadikan sebagai `key` di dalam `LaunchedEffect(number)`. Setiap kali nilai `number` pertama kali diinisialisasi atau nilainya berubah, `LaunchedEffect` akan terpicu untuk menjalankan kodenya.

```
LaunchedEffect( key1 = number) {  
    snackbarHostState.showSnackbar( message = "Number $number is  
        shown!")  
}
```

2. Klik button "Double" satu kali,
 - a. Apa yang terjadi?

Angka pada tampilan Text berubah dari 1 menjadi 2 Lalu Snackbar baru muncul di bagian bawah layar dengan pesan: "Number 2 is shown!"

b. Mengapa hal tersebut terjadi? Jelaskan!

- **Perubahan Angka (1 → 2):**

Saat tombol "Double" diklik, callback `onButtonClick` mengeksekusi perintah `{ number *= 2 }`. Karena `number` adalah variabel state (`rememberSaveable`), perubahan nilai ini memicu rekomposisi pada UI sehingga komponen Text memperbarui tampilannya menjadi 2.

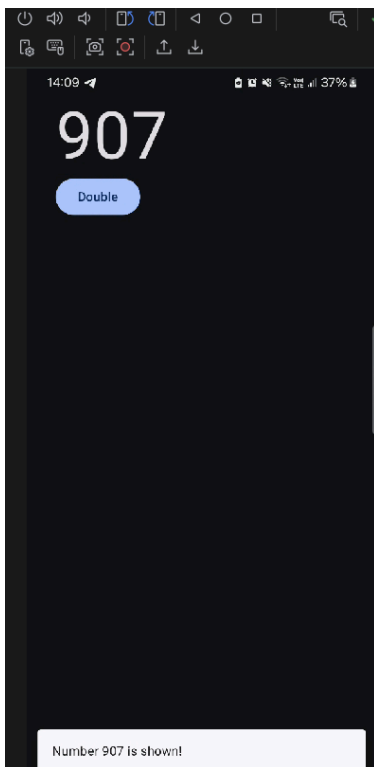
- **Kemunculan Snackbar Baru:**

Variabel `number` dipasang sebagai key pada `LaunchedEffect(number)`. Ketika nilai `number` berubah dari 1 ke 2, Compose mendeteksi adanya perubahan kunci tersebut dan secara otomatis membatalkan effect sebelumnya lalu memicu ulang (re-trigger) blok `LaunchedEffect`. Akibatnya, perintah `snackbarHostState.showSnackbar("Number 2 is shown!")` kembali dieksekusi dengan nilai angka yang baru (2).

Aktivitas Percobaan 2

1. Jalankan program,

a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?



b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

- **Angka pada layar:** Berubah secara otomatis dari 1 menjadi angka acak antara 100 sampai 999 (misal: 543).
- **Snackbar:** Muncul Snackbar baru yang memperbarui pesan menjadi: "Number 543 is shown!" (sesuai angka acak yang dihasilkan).

- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?

Pemicu Perubahan Angka (1 → Angka Acak):

LaunchedEffect(Unit) mengeksekusi coroutine asynchronous saat pertama kali UI dimuat. Setelah menunggu penundaan 2 detik (delay(2000) di Repo.getData()), fungsi tersebut mengembalikan angka acak dan memperbarui variabel state number.

```
LaunchedEffect( key1 = Unit) {  
    number = Repo.getData()  
}
```

Pemicu Pesan SnackBar Baru:

Perubahan nilai number dari 1 menjadi angka acak memicu ulang (re-trigger) LaunchedEffect(number) karena number dipasang sebagai kuncinya (key).

```
LaunchedEffect( key1 = number) {  
    snackbarHostState.showSnackBar( message = "Number $number is  
    shown!")  
}
```

2. Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect,

- a. Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?

Fitur	LaunchedEffect(Unit)	LaunchedEffect(state)
Kunci (Key)	Unit (nilai konstan/tetap).	Variables/State dinamis (misal: number).
Frekuensi Eksekusi	Hanya 1 kali sepanjang lifecycle Composable berada di layar.	Bisa berkali-kali, setiap kali nilai state berubah.
Tujuan Penggunaan	Untuk tugas one-shot (sekali jalan) saat UI pertama dimuat.	Untuk merespons perubahan data secara dinamis (reactive effect).

- b. Kapan operasi di dalam LaunchedEffect(state) dengan LaunchedEffect(Unit) dijalankan?

- i. LaunchedEffect(Unit) dijalankan pada:

1. Hanya saat Initial Composition (pertama kali komponen UI dimasukkan/digambar di layar).
2. Contoh penggunaan: Memuat data awal dari server/repository saat halaman dibuka (Repo.getData()).

- ii. LaunchedEffect(state) dijalankan pada:

1. Pertama kali komponen dirender (Initial Composition).
2. Setiap kali nilai state yang menjadi key-nya berubah.

3. (Jika ada coroutine lama yang belum selesai, coroutine lama akan dibatalkan/cancelled dan digantikan dengan coroutine baru dengan nilai state terbaru).